

ShopEZ: E-commerce Application

INTRODUCTION

ShopEZ is your one-stop destination for effortless online shopping. With a user-friendly interface and a comprehensive product catalog, finding the perfect items has never been easier. Seamlessly navigate through detailed product descriptions, customer reviews, and available discounts to make informed decisions. Enjoy a secure checkout process and receive instant order confirmation. For sellers, our robust dashboard provides efficient order management and insightful analytics to drive business growth. Experience the future of online shopping with ShopEZ today.

SCENARIO:

Sarah's Birthday Gift

Sarah, a busy professional, is scrambling to find the perfect birthday gift for her best friend, Emily. She knows Emily loves fashion accessories, but with her hectic schedule, she hasn't had time to browse through multiple websites to find the ideal present. Feeling overwhelmed, Sarah turns to ShopEZ to simplify her search.

1. Effortless Product Discovery: Sarah opens ShopEZ and navigates to the fashion accessories category. She's greeted with a diverse range of options, from chic handbags to elegant jewelry. Using the filtering options, Sarah selects "bracelets" and refines her search based on Emily's preferred style and budget.

2. Personalized Recommendations: As Sarah scrolls through the curated selection of bracelets, she notices a section labeled "Recommended for You." Intrigued, she clicks on it and discovers a stunning gold bangle that perfectly matches Emily's taste. Impressed by the personalized recommendation, Sarah adds it to her cart.

3. Seamless Checkout Process: With the bracelet in her cart, Sarah proceeds to checkout. She enters Emily's address as the shipping destination and selects her

preferred payment method. Thanks to ShopEZ's secure and efficient checkout process, Sarah completes the transaction in just a few clicks.

4. Order Confirmation: Moments after placing her order, Sarah receives a confirmation email from ShopEZ. Relieved to have found the perfect gift for Emily, she eagerly awaits its arrival.

5. Efficient Order Management for Sellers: Meanwhile, on the other end, the seller of the gold bangle receives a notification of Sarah's purchase through ShopEZ's seller dashboard. They quickly process the order and prepare it for shipment, confident in ShopEZ's streamlined order management system.

6. Celebrating with Confidence: On Emily's birthday, Sarah presents her with the beautifully packaged bracelet, knowing it was chosen with care and thoughtfulness. Emily's eyes light up with joy as she adorns the bracelet, grateful for Sarah's thoughtful gesture.

In this scenario, ShopEZ proves to be the perfect solution for Sarah's busy lifestyle, offering a seamless and personalized shopping experience. From effortless product discovery to secure checkout and efficient order management, ShopEZ simplifies the entire process, allowing Sarah to celebrate Emily's birthday with confidence and ease.

SYSTEM REQUIREMENTS :

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 [Download Node.js](#)
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.

- **MongoDB:** A NoSQL database used to store structured and unstructured data to users, admin, product, orders and cart.
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub:** For version control and collaborative development.

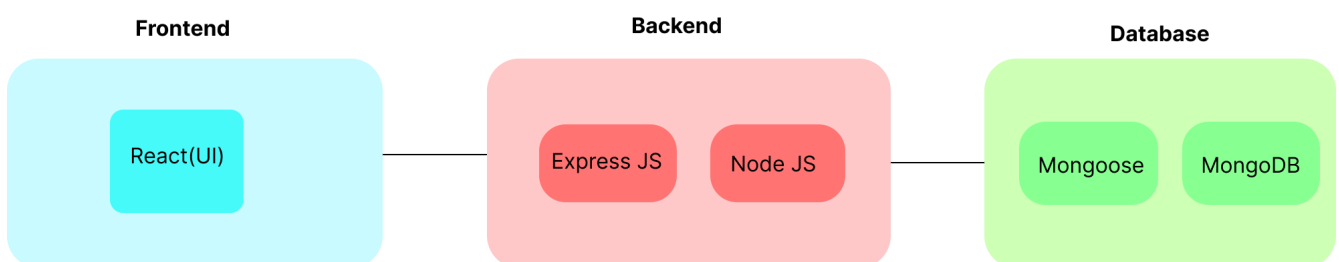
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE :

TECHNICAL ARCHITECTURE:



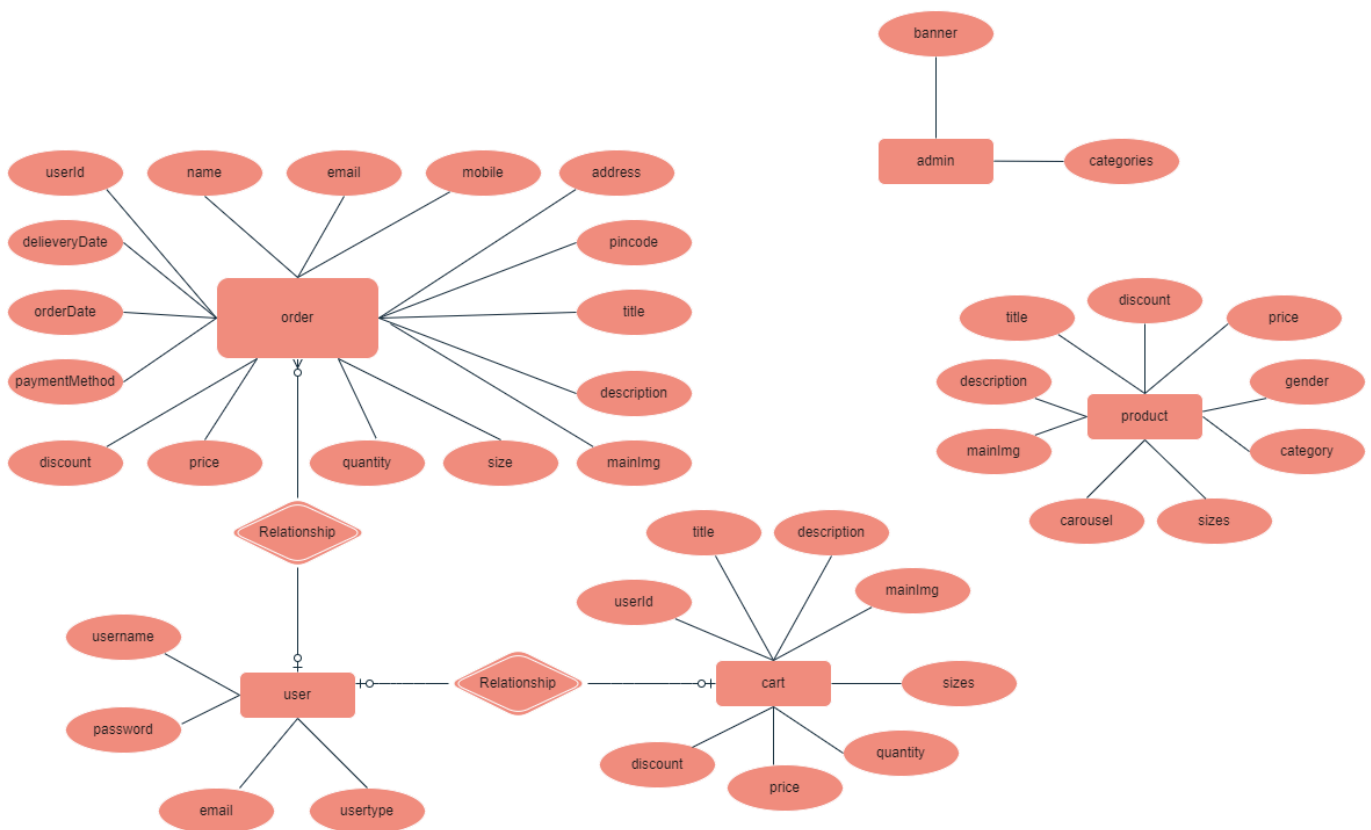
In this architecture diagram:

- The frontend is represented by the "Frontend" section, including user interface components such as User Authentication, Cart, Products, Profile, Admin dashboard, etc.,

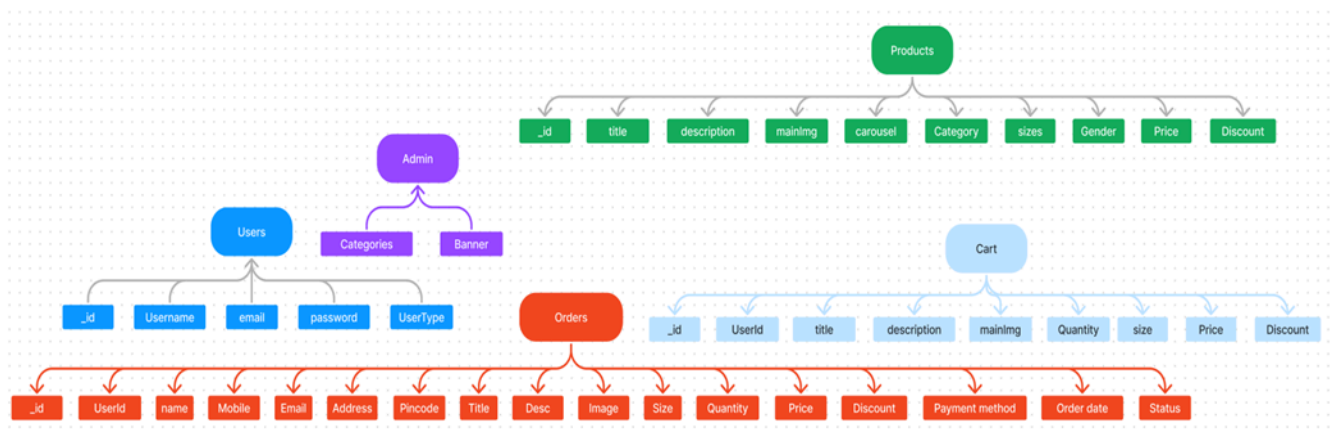
- The backend is represented by the "Backend" section, consisting of API endpoints for Users, Orders, Products, etc., It also includes Admin Authentication and an Admin Dashboard.
- The Database section represents the database that stores collections for Users, cart, Orders and Product.

ER DIAGRAM:

ER-MODEL



- The Database section represents the database that stores collections for Users, Admin, Cart, Orders and products.



The ShopEZ ER-diagram represents the entities and relationships involved in an e-commerce system. It illustrates how users, products, cart, and orders are interconnected. Here is a breakdown of the entities and their relationships:

USER: Represents the individuals or entities who are registered in the platform.

Admin: Represents a collection with important details such as Banner image and Categories.

Products: Represents a collection of all the products available in the platform.

Cart: This collection stores all the products that are added to the cart by users. Here, the elements in the cart are differentiated by the user Id.

Orders: This collection stores all the orders that are made by the users in the platform.

FEATURES:

1. **Comprehensive Product Catalog:** ShopEZ boasts an extensive catalog of products, offering a diverse range of items and options for shoppers. You can effortlessly explore and discover various products, complete with detailed descriptions, customer reviews, pricing, and available discounts, to find the perfect items for your needs.
2. **Shop Now Button:** Each product listing features a convenient "Shop Now" button. When you find a product that aligns with your preferences, simply click on the button to initiate the purchasing process.
3. **Order Details Page:** Upon clicking the "Shop Now" button, you will be directed to an order details page. Here, you can provide relevant information such as your shipping address, preferred payment method, and any specific product requirements.
4. **Secure and Efficient Checkout Process:** ShopEZ guarantees a secure and efficient checkout process. Your personal information will be handled with the utmost security, and we strive to make the purchasing process as swift and trouble-free as

possible.

5. **Order Confirmation and Details:** After successfully placing an order, you will receive a confirmation notification. Subsequently, you will be directed to an order details page, where you can review all pertinent information about your order, including shipping details, payment method, and any specific product requests you specified.

In addition to these user-centric features, ShopEZ provides a robust seller dashboard, offering sellers an array of functionalities to efficiently manage their products and sales. With the seller dashboard, sellers can add and oversee multiple product listings, view order history, monitor customer activity, and access order details for all purchases.

ShopEZ is designed to elevate your online shopping experience by providing a seamless and user-friendly way to discover and purchase products. With our efficient checkout process, comprehensive product catalog, and robust seller dashboard, we ensure a convenient and enjoyable online shopping experience for both shoppers and sellers alike.

ROLES AND RESPONSIBILITIES :

USER: Represents the individuals or entities who are registered in the platform.

Admin: Represents a collection with important details such as Banner image and Categories.

Products: Represents a collection of all the products available in the platform.

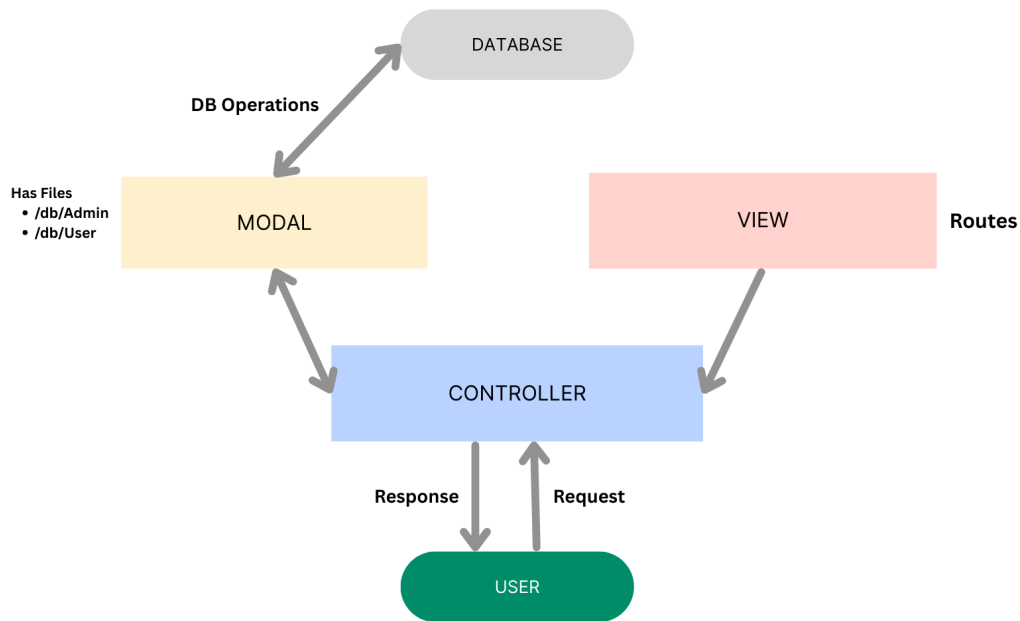
Cart: This collection stores all the products that are added to the cart by users. Here, the elements in the cart are differentiated by the user Id.

Orders: This collection stores all the orders that are made by the users in the platform.

User Flow

- Users start by registering for an account.
- After registration, they can log in with their credentials.
- Once logged in, they can check for the available products in the platform.
- Users can add the products they wish to their carts and order.
- They can then proceed by entering address and payment details.
- After ordering, they can check them in the profile section.

MVC Pattern



The Shop-Ez backend application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.

PROJECT SETUP AND CONFIGURATION

● Creating project folder

1. Create a new folder with your <project name>.
2. Inside that folder create two new folders.
3. Name one as ****Client****.
4. Name another one as ****Server****.
5. Now open that folder in VS Code.

● Client setup (installing react app)

- Open the Client folder in the terminal of VScode.
 - `npm create vite@latest . -- --template react`
- Select React framework from the given options.
 - Select a framework:
 - | React
- Select JavaScript variant from the given options.
 - Select a variant:
 - | JavaScript
- Now lets navigate to the client folder by giving the following command.
 - `cd client`
- To install all the packages run the following command.
 - `npm install`
- To start the React server type the following command.
 - `npm run dev`

● Server setup (npm init)

- Open Server folder in terminal of VScode.
 - `npm init -y`
- Create files:
 - `server.js`

- Create folders:
 - models
 - controllers
 - routes

BACKEND DEVELOPMENT:

<https://drive.google.com/file/d/1Gi-Np-oh4WP50p8fyJHFhMjZ0ULjb5pP/view?usp=sharing>

1.Setup express server:

- Create index.js file.
- Create an express server on your desired port number.
- Define API's

2. Database Configuration:

- Set up a MongoDB database either locally or using a cloud-based MongoDB service like MongoDB Atlas or use locally with MongoDB compass.
- Create a database and define the necessary collections for admin, users, products, orders and other relevant data.

3. Create Express.js Server:

- Set up an Express.js server to handle HTTP requests and serve API endpoints.
- Configure middleware such as body-parser for parsing request bodies and cors for handling cross-origin requests.

4. Define API Routes:

- Create separate route files for different API functionalities such as users, orders, and authentication.
- Define the necessary routes for listing products, handling user registration

and login, managing orders, etc.

- Implement route handlers using Express.js to handle requests and interact with the database.

5. Implement Data Models:

- Define Mongoose schemas for the different data entities like products, users, and orders.
- Create corresponding Mongoose models to interact with the MongoDB database.
- Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations.

6. User Authentication:

- Create routes and middleware for user registration, login, and logout.
- Set up authentication middleware to protect routes that require user authentication.

7. Handle new products and Orders:

- Create routes and controllers to handle new product listings, including fetching products data from the database and sending it as a response.
- Implement ordering(buy) functionality by creating routes and controllers to handle order requests, including validation and database updates.

8. Admin Functionality:

- Implement routes and controllers specific to admin functionalities such as adding products, managing user orders, etc.
- Add necessary authentication and authorization checks to ensure only authorized admins can access these routes.

9. Error Handling:

- Implement error handling middleware to catch and handle any errors that occur during the API requests.
- Return appropriate error responses with relevant error messages and

HTTP status codes.

DATABASE DEVELOPMENT :

The screenshot shows a Visual Studio Code editor with a project named 'SHOPEZ'. The Explorer sidebar on the left shows the file structure: 'client', 'server', 'node_modules', '.env', 'index.js', 'package-lock.json', and 'package.json'. The main editor area displays the 'index.js' file in the 'server' directory. The code imports 'express', 'mongoose', 'cors', and 'dotenv'. It configures 'dotenv' to use the '.env' file, sets up an Express app with JSON and CORS middleware, and listens on port 3001. A 'connectToMongo' function is defined to connect to a MongoDB database using 'process.env.DRIVER_LINK'. The terminal at the bottom shows the command 'node index.js' being executed, with output messages: 'App server is running on port 3001', 'bad auth : authentication failed', and 'Connected to your MongoDB database successfully'.

```
1 import express from "express";
2 import mongoose from "mongoose";
3 import cors from "cors";
4 import dotenv from "dotenv";
5
6 dotenv.config({ path: "./.env" });
7
8 const app = express();
9 app.use(express.json());
10 app.use(cors());
11
12 app.listen(3001, () => {
13   console.log("App server is running on port 3001");
14 });
15
16 const MongoUri = process.env.DRIVER_LINK;
17 const connectToMongo = async () => {
18   try {
19     await mongoose.connect(MongoUri);
20     console.log("Connected to your MongoDB database successfully");
21   } catch (error) {
22     console.log(error.message);
23   }
24 };
25
26 connectToMongo();
```

PS D:\shopEZ> cd server
PS D:\shopEZ\server> node index.js
App server is running on port 3001
bad auth : authentication failed
PS D:\shopEZ\server> node index.js
App server is running on port 3001
Connected to your MongoDB database successfully

Schema use-case:

1. User Schema:

- Schema: userSchema
- Model: 'User'
- The User schema represents the user data and includes fields such as username, email, and password.
- It is used to store user information for registration and authentication purposes.
- The email field is marked as unique to ensure that each user has a unique email

address

2. Product Schema:

- Schema: productSchema
- Model: 'Product'
- The Product schema represents the data of all the products in the platform.
- It is used to store information about the product details, which will later be useful for ordering .

3. Orders Schema:

- Schema: ordersSchema
- Model: 'Orders'
- The Orders schema represents the orders data and includes fields such as
userId, product Id, product name, quantity, size, order date, etc.,
- It is used to store information about the orders made by users.
- The user Id field is a reference to the user who made the order.

4. Cart Schema:

- Schema: cartSchema
- Model: 'Cart'
- The Cart schema represents the cart data and includes fields such as userId,
product Id, product name, quantity, size, order date, etc.,
- It is used to store information about the products added to the cart
by users. • The user Id field is a reference to the user who has the
product in cart.

5. Admin Schema:

- Schema: adminSchema
 - Model: 'Admin'
 - The admin schema has essential data such as categories, banner.

Code Explanation:

Schemas:

Now let us define the required schemas

```

JS Schema.js X
server > JS Schema.js > [0] productSchema
1  import mongoose from "mongoose";
2
3  const userSchema = new mongoose.Schema({
4      username: {type: String},
5      password: {type: String},
6      email: {type: String},
7      usertype: {type: String}
8  });
9
10 const adminSchema = new mongoose.Schema({
11     banner: {type: String},
12     categories: {type: Array}
13 });
14
15 const productSchema = new mongoose.Schema({
16     title: {type: String},
17     description: {type: String},
18     mainImg: {type: String},
19     carousel: {type: Array},
20     sizes: {type: Array},
21     category: {type: String},
22     gender: {type: String},
23     price: {type: Number},
24     discount: {type: Number}
25 })
26

```

```

JS Schema.js X
server > JS Schema.js > [0] productSchema
27 const orderSchema = new mongoose.Schema({
28     userId: {type: String},
29     name: {type: String},
30     email: {type: String},
31     mobile: {type: String},
32     address: {type: String},
33     pincode: {type: String},
34     title: {type: String},
35     description: {type: String},
36     mainImg: {type: String},
37     size: {type: String},
38     quantity: {type: Number},
39     price: {type: Number},
40     discount: {type: Number},
41     paymentMethod: {type: String},
42     orderDate: {type: String},
43     deliveryDate: {type: String},
44     orderStatus: {type: String, default: 'order placed'}
45 })
46
47 const cartSchema = new mongoose.Schema({
48     userId: {type: String},
49     title: {type: String},
50     description: {type: String},
51     mainImg: {type: String},
52     size: {type: String},
53     quantity: {type: String},
54     price: {type: Number},
55     discount: {type: Number}
56 })
57
58
59 export const User = mongoose.model('users', userSchema);
60 export const Admin = mongoose.model('admin', adminSchema);
61 export const Product = mongoose.model('products', productSchema);
62 export const Orders = mongoose.model('orders', orderSchema);
63 export const Cart = mongoose.model('cart', cartSchema);
64

```

FRONT-END DEVELOPMENT :

https://drive.google.com/file/d/18l14DodtZvhB_u72CtrDibpV5vxiq4_J/view?usp=sharing

1. Setup React Application:

- Create a React app in the client folder.
- Install required libraries
- Create required pages and components and add routes.

2.Design UI components:

- Create Components.
- Implement layout and styling.
- Add navigation.

3.Implement frontend logic:

- Integration with API endpoints.
- Implement data binding.

PROJECT EXECUTION

Steps for Project Execution:

Step 1: Set Up the Frontend (React App):

- a) Open a terminal and navigate into the client folder:

cd client

- b) Once installation is done, start the React development server:

npm run dev

- c) The app should now be running on:

<http://localhost:5173>

Step 2: Set Up the Backend (Express Server)

- a) Open a new terminal tab/window or split the terminal.

- b) Navigate into the server folder:

cd ../server

Step 3: Configure Environment Variables

a) Inside the server folder, create a new file named `.env` (no file extension).

b) In that `.env` file, add your MongoDB connection string:

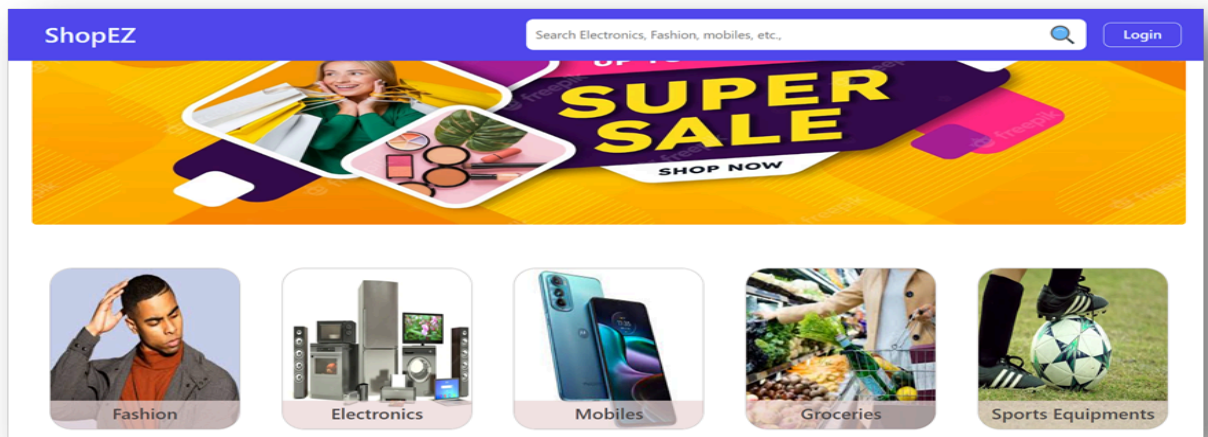
`MONGO_URI=mongodb://localhost:27017/<— dbname—>`

Step 4: Start the Backend Server:

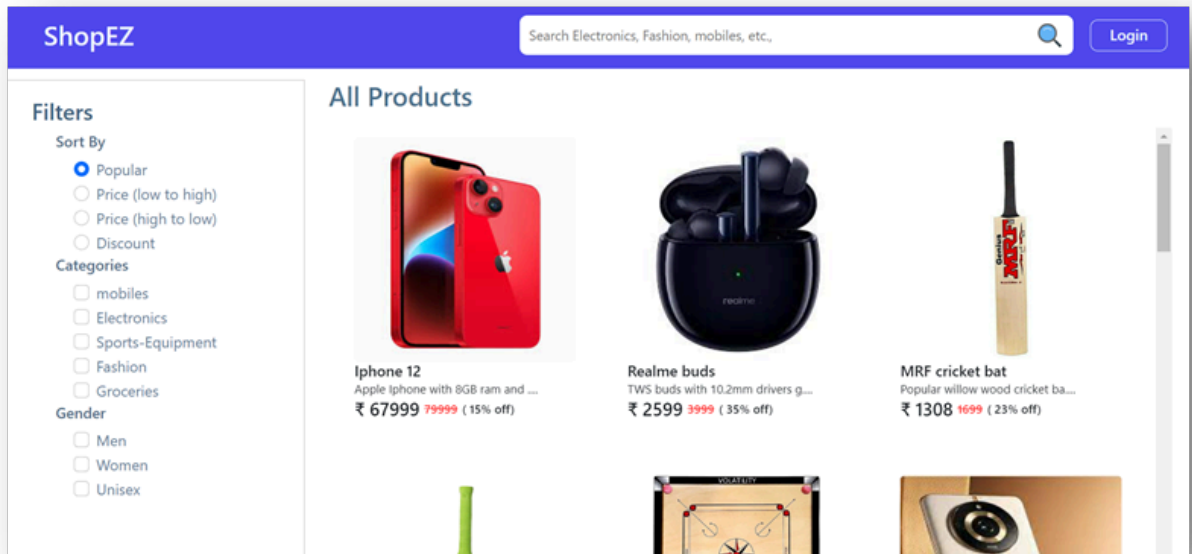
- Inside the same server folder, run the backend server using:
`nodemon index.js`
- The server should start on:
<http://localhost:8000>

OUTPUT SCREENSHOTS :

Landing page : This is the landing page of the application .



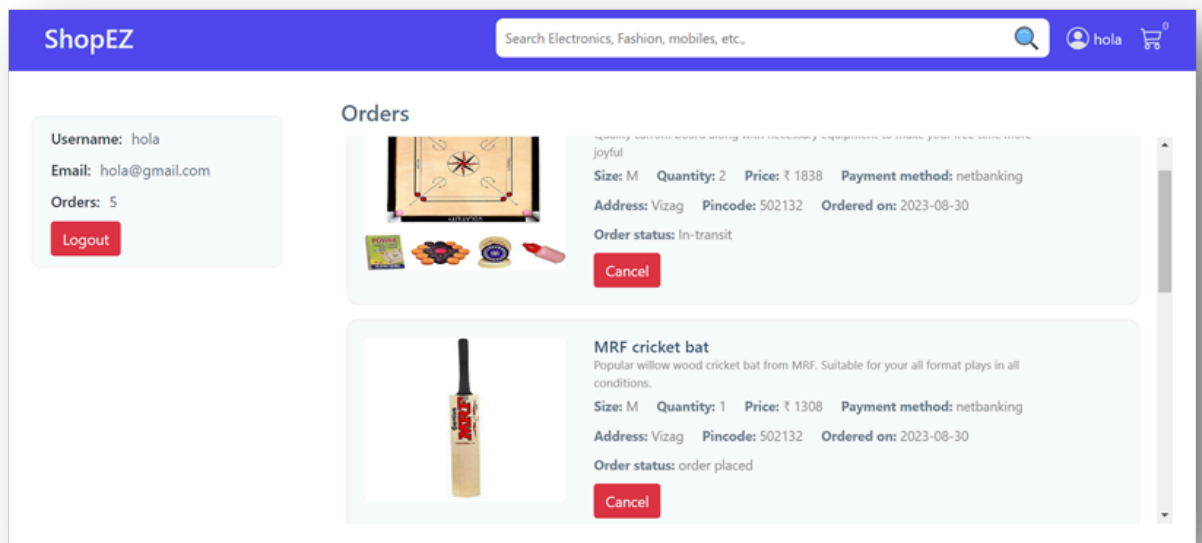
- **Products :** This is the products page . It lists all the available products.



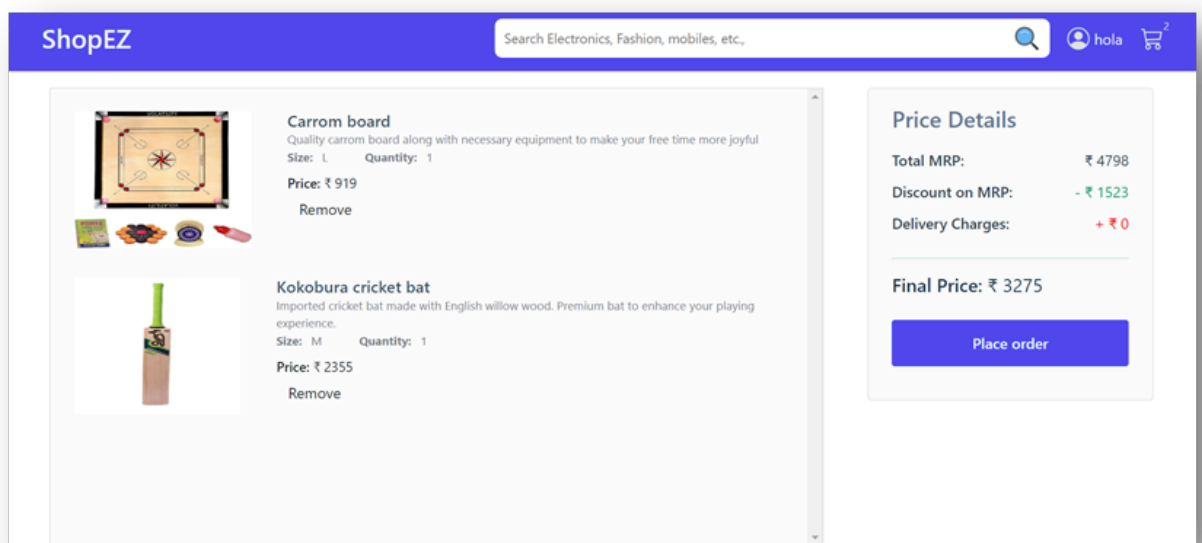
- **Authentication:** This is the registration page where we need to register the application by entering the inputs.

The screenshot shows the ShopEZ registration page. It features a blue header with the ShopEZ logo, a search bar, and a "Login" button. The main content area contains a "Register" form with the following fields: "Username", "Email address", "Password", and a "User type" dropdown menu. The dropdown menu is open, showing "User type" as the selected option, with "Admin" and "Customer" as other options. Below the form is a link that says "Already registered? Login".

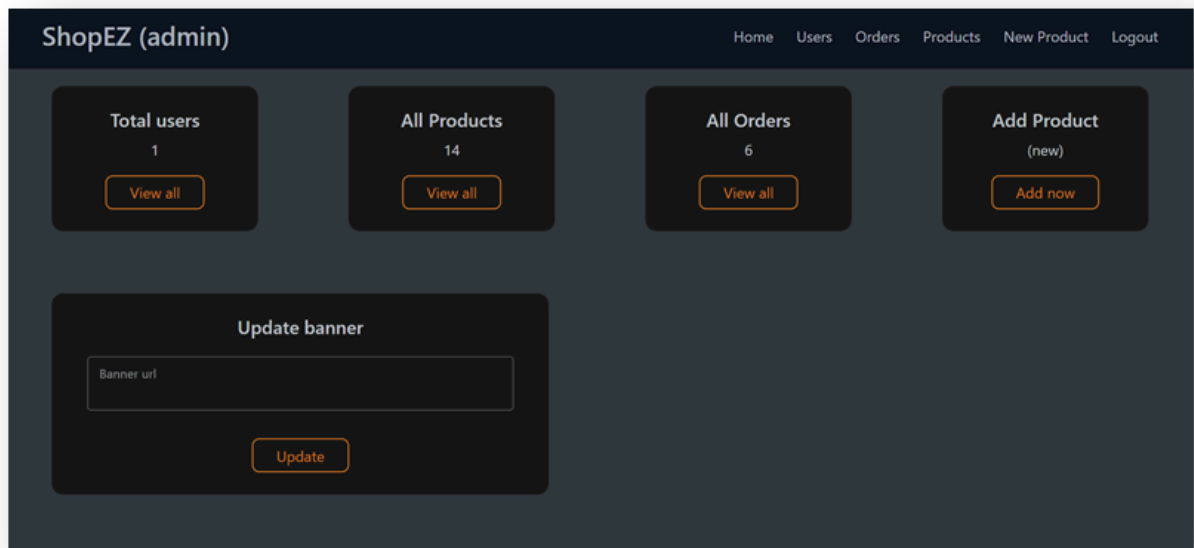
- **User Profile :** This is the user profile page . It displays the users details and orders.



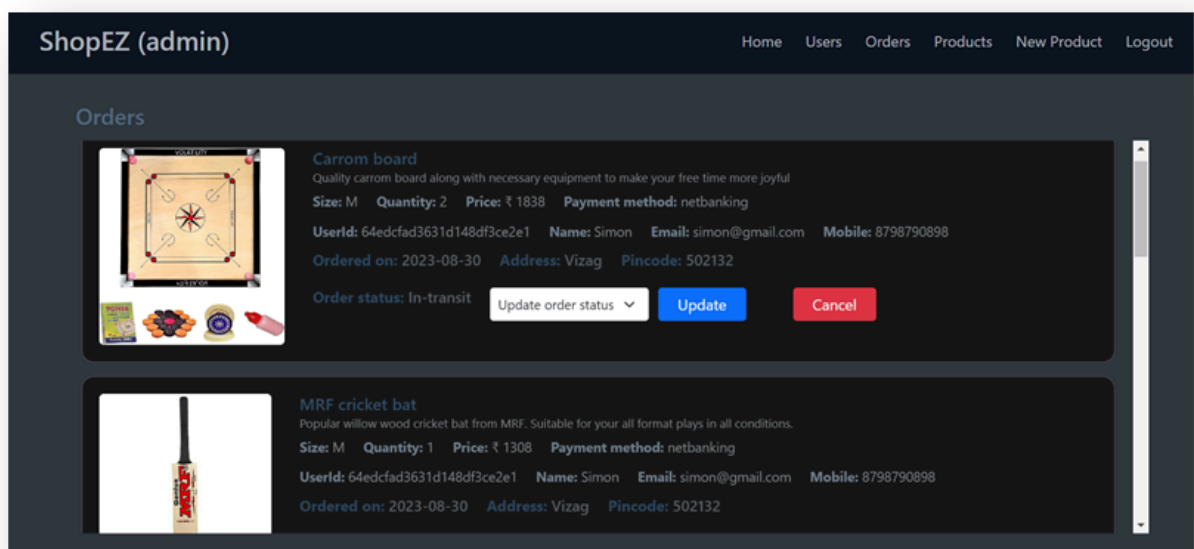
- **Cart :** This is the cart page , where users can add products to the cart .



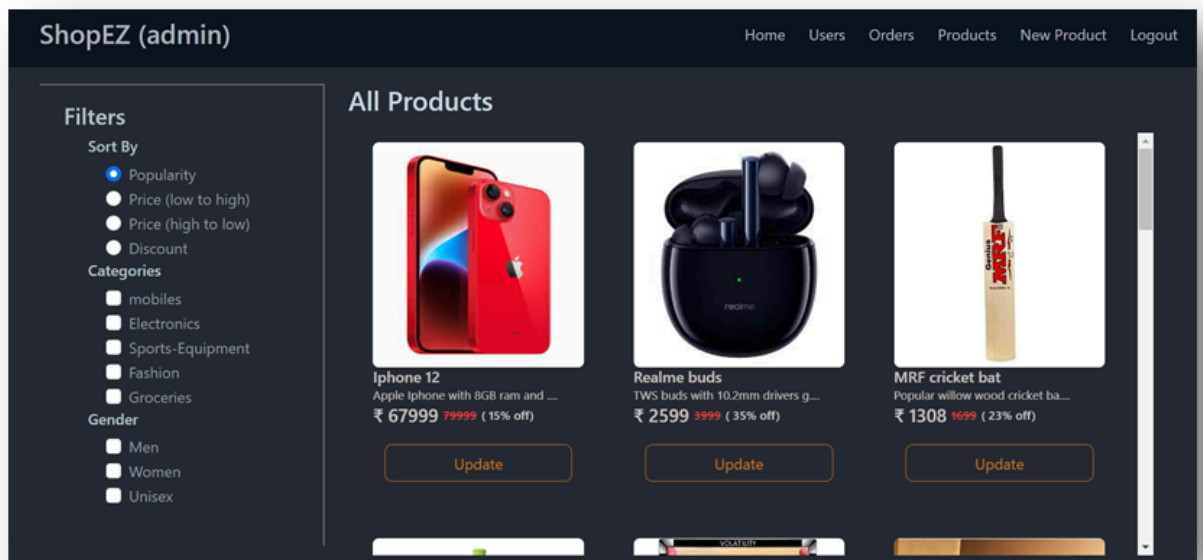
- **Admin dashboard :** This is the admin dashboard page .



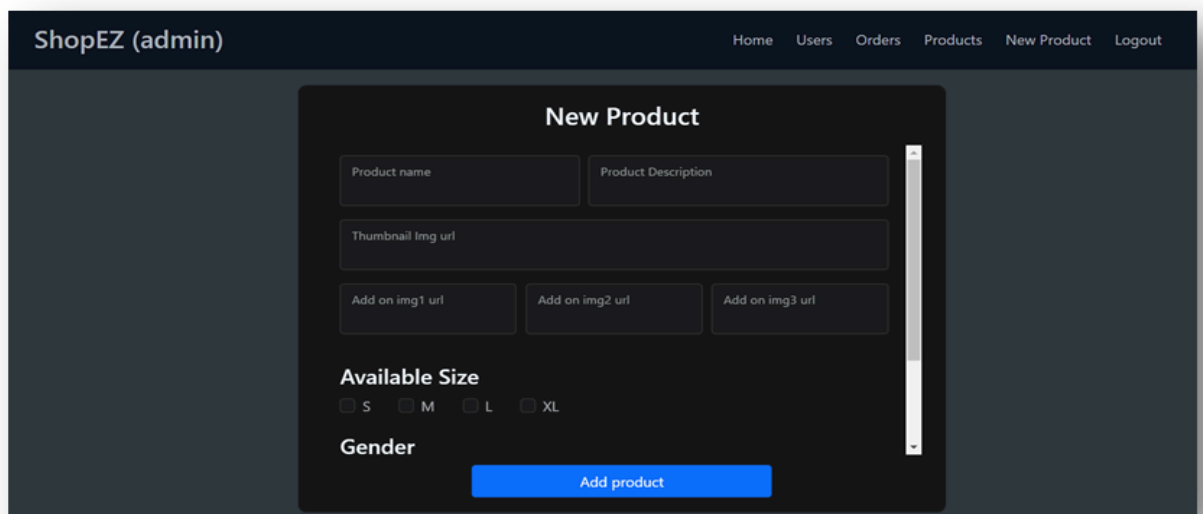
- **All Orders :** This is all orders page , where admin can see all the orders that are placed .



- **All Products** : This is the all products page , where admins can display all the available products .



- **New Product Page:** This is the new product page where admin can add new products.



DemoVideo: :  shopEZ MERN demo (1).mp4

Code :  code